

PROJECT 3 • APPLIED INFRASTRUCTURE AS CODE

# Serverless Architecture with Terraform

*Lambda, API Gateway, S3, and DynamoDB — provisioned end-to-end and tested with a live request.*



Lambda



API Gateway



S3



DynamoDB

## OVERVIEW

# What we'll build

A complete serverless backend: a REST endpoint that writes to a database, backed by four AWS services.



**AWS Lambda**

Python function that writes an item to DynamoDB.



**API Gateway**

Exposes the Lambda function as a REST API.



**Amazon S3**

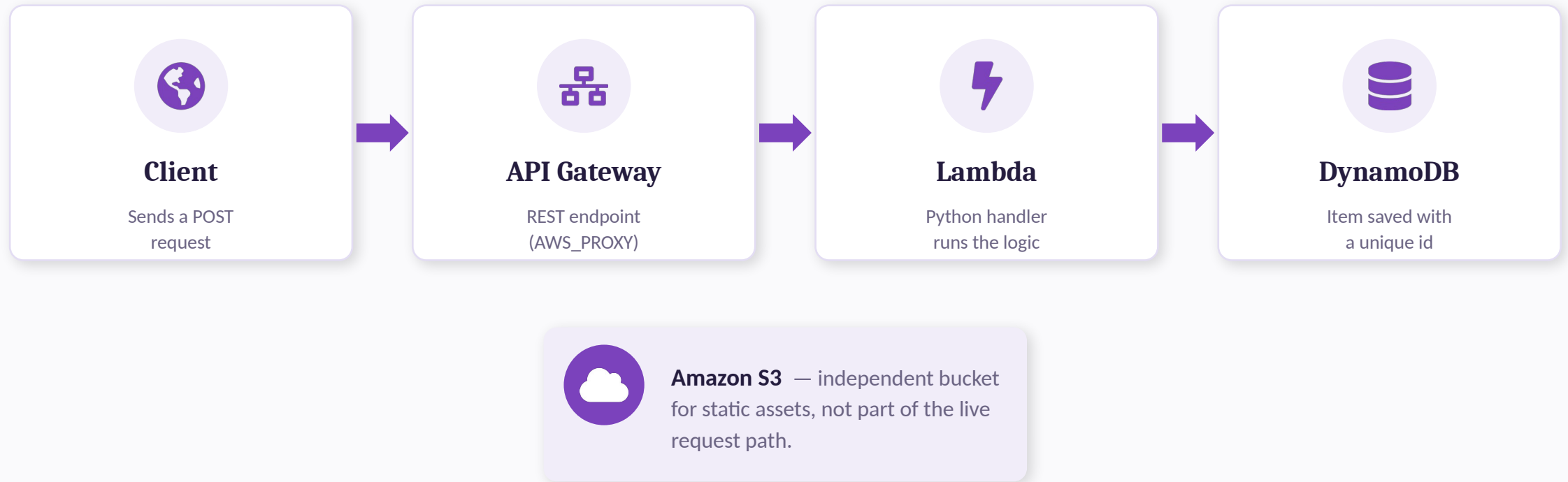
Stores static assets for the application.



**DynamoDB**

Serverless, pay-per-request data storage.

# How a request flows through the system



**End to end:** the client never talks to Lambda or DynamoDB directly — API Gateway is the single, managed front door.

# Folder structure

Each AWS service gets its own module folder, kept together by the root configuration.

```
terraform-serverless/  
├── main.tf  
├── variables.tf  
├── outputs.tf  
├── lambda/  
│   ├── lambda_function.py  
│   ├── requirements.txt  
│   └── main.tf  
├── api_gateway/  
│   └── main.tf  
├── s3/  
│   └── main.tf  
├── dynamodb/  
│   └── main.tf
```



## Root config

main.tf, variables.tf, outputs.tf wire the modules together.



## lambda/

Function code plus its own Terraform resource.



## api\_gateway/

REST API, resource, method & integration.



## s3/ & dynamodb/

One focused module per storage service.

## Store static assets in S3

```
resource "aws_s3_bucket" "my_bucket" {  
  bucket = var.bucket_name  
}  
  
resource "aws_s3_bucket_public_access_block"  
"public_access" {  
  bucket = aws_s3_bucket.my_bucket.id  
  
  block_public_acls       = false  
  block_public_policy     = false  
  ignore_public_acls     = false  
  restrict_public_buckets = false  
}
```



### Heads up

This access block **disables** every public-access restriction — fine for a quick demo bucket, but review it before storing anything sensitive. In production, keep these flags **true** unless the bucket must serve public files.

## Store application data in DynamoDB

```
resource "aws_dynamodb_table" "my_table" {  
  name           = var.table_name  
  billing_mode    = "PAY_PER_REQUEST"  
  hash_key        = "id"  
  
  attribute {  
    name = "id"  
    type = "S"  
  }  
}
```

### Why PAY\_PER\_REQUEST

- No capacity planning — scales automatically
- You pay only for reads/writes you use
- Ideal for spiky or unpredictable traffic
- hash\_key "id" is the table's partition key



This table is what the Lambda function writes to in the next step — its name is passed in as **TABLE\_NAME**.

## Wire up the Lambda resource

```
resource "aws_lambda_function" "my_lambda" {  
  filename      = "lambda_function.zip"  
  function_name = "my_lambda_function"  
  role          = aws_iam_role.lambda_exec.arn  
  handler       = "lambda_function.lambda_handler"  
  runtime       = "python3.9"  
  
  environment {  
    variables = {  
      TABLE_NAME = aws_dynamodb_table.my_table.name  
    }  
  }  
}
```

### Key arguments

#### filename

Path to the zipped deployment package.

#### role

IAM role granting DynamoDB write access.

#### handler

file.function that Lambda invokes.

#### environment

Passes the table name in at runtime.

# What the function actually does

```
import json
import boto3
import os
import uuid

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['TABLE_NAME'])

def lambda_handler(event, context):
    item = {
        'id': str(uuid.uuid4()),
        'message': 'Hello from Lambda!'
    }
    table.put_item(Item=item)

    return {
        'statusCode': 200,
        'body': json.dumps(item)
    }
```

## Line by line

- 1 Connect to the table named in the TABLE\_NAME environment variable.
- 2 Build an item with a fresh UUID as its id.
- 3 Write the item to DynamoDB with put\_item.
- 4 Return HTTP 200 with the item as JSON — API Gateway relays this to the caller.



## Expose the Lambda as a REST API

```
resource "aws_api_gateway_rest_api" "my_api" {
  name      = "ServerlessAPI"
  description = "API Gateway for Lambda"
}

resource "aws_api_gateway_resource" "lambda_resource" {
  rest_api_id = aws_api_gateway_rest_api.my_api.id
  parent_id   =
aws_api_gateway_rest_api.my_api.root_resource_id
  path_part   = "lambda"
}
```

```
resource "aws_api_gateway_method" "lambda_post" {
  rest_api_id = aws_api_gateway_rest_api.my_api.id
  resource_id =
aws_api_gateway_resource.lambda_resource.id
  http_method = "POST"
  authorization = "NONE"
}

resource "aws_api_gateway_integration"
"lambda_integration" {
  rest_api_id = aws_api_gateway_rest_api.my_api.id
  resource_id =
aws_api_gateway_resource.lambda_resource.id
  http_method =
aws_api_gateway_method.lambda_post.http_method
  integration_http_method = "POST"
  type = "AWS_PROXY"
  uri = aws_lambda_function.my_lambda.invoke_arn
}
```

## Ship it, then check what you have



```
$ terraform init  
$ terraform apply -auto-approve
```

### ✓ What we have built



Lambda function that writes data to DynamoDB



API Gateway that exposes the Lambda function



S3 bucket for storing static assets



DynamoDB table for persistent storage

## VERIFICATION

# Test the live Serverless API

1

### Get the invoke URL

terraform apply prints the API Gateway URL — or find it under API Gateway ► Stages ► Invoke URL.

2

### Send a POST request

Trigger the Lambda with cURL, Postman, or any API client.

3

### Verify in DynamoDB

Check DynamoDB ► Tables ► my\_table ► Items for the new record.



```
$ curl -X POST \
https://abcdefghij.execute-api.us-east-1.amazonaws.com/prod/lambda
```



```
{
  "id": "f68f1d27-9b7d-423d-b3ab-7e1c784b4e79",
  "message": "Hello from Lambda!"
}
```

**Postman:** set method to POST, paste the invoke URL, hit Send.



A response like this — plus a matching new row in DynamoDB — confirms the whole chain works.

## PROJECT 3 SUMMARY

# A working serverless backend, start to finish



### S3 bucket

Static asset storage, provisioned in its own module.



### DynamoDB table

PAY\_PER\_REQUEST billing keyed on id.



### Lambda function

Python handler that writes a UUID-keyed item.



### API Gateway

AWS\_PROXY integration exposing a POST endpoint.